**MLE ASSIGNMENT - 4**

**ASSIGNMENT**

**On**

# LINEAR & POLYNOMIAL REGRESSION - AUTO MPG ANALYSIS

Submitted to

MIT ADT University, Pune

in partial fulfillment of the requirements for the Assignment of the course

Machine Learning Essentials

Submitted by

**PRANAV WAGH ADT23SOCB0752**

**Roll No : 23**

Under the Guidance of

**Prof. Vijaya Patil**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING**
**MIT SCHOOL OF COMPUTING, MIT ADT UNIVERSITY**
**RAJBAUG, LONI KALBHOR, PUNE – 412201**

**THIRD YEAR - SOC** **A.Y 2025-26** **SEMESTER-II**

**MLE ASSIGNMENT - 4**

**Scenario:-**

**An automobile research company wants to predict miles-per-gallon (MPG) based on engine characteristics such as horsepower, weight, displacement, and cylinders.**
**The analyst must: build linear regression and polynomial regression models, study underfitting and overfitting, observe learning and validation curves, explain performance differences.**

**Aim:-**

To develop and compare **linear and polynomial regression models** for predicting automobile fuel efficiency using the Auto MPG dataset.

## Question 1 — Load Dataset

Load Auto MPG dataset and display: • first 10 rows • dataset shape • column names

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, learning_curve, validation_curve
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import warnings
warnings.filterwarnings('ignore')

# Load dataset, treating '?' as NaN
df = pd.read_csv('D:/MLE/Assignment_4/auto-mpg.csv', na_values='?')

# Display first 10 rows
print("First 10 rows:")
print(df.head(10))

# Dataset shape
print("\nDataset shape:", df.shape)

# Column names
print("\nColumn names:", df.columns.tolist())
```

```
First 10 rows:
    mpg  cylinders  displacement  horsepower  weight  acceleration  \
0  18.0          8         307.0       130.0    3504          12.0
1  15.0          8         350.0       165.0    3693          11.5
2  18.0          8         318.0       150.0    3436          11.0
3  16.0          8         304.0       150.0    3433          12.0
4  17.0          8         302.0       140.0    3449          10.5
5  15.0          8         429.0       198.0    4341          10.0
6  14.0          8         454.0       220.0    4354           9.0
7  14.0          8         440.0       215.0    4312           8.5
8  14.0          8         455.0       225.0    4425          10.0
9  15.0          8         390.0       190.0    3850           8.5

   model year  origin                   car name
0          70       1  chevrolet chevelle malibu
1          70       1          buick skylark 320
2          70       1         plymouth satellite
3          70       1              amc rebel sst
4          70       1                ford torino
5          70       1           ford galaxie 500
6          70       1           chevrolet impala
7          70       1          plymouth fury iii
8          70       1           pontiac catalina
9          70       1         amc ambassador dpl

Dataset shape: (398, 9)

Column names: ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model year', 'origin', 'car name']
```

**MLE ASSIGNMENT - 4**

## Question 2 — Data Cleaning

Handle missing values in mpg and horsepower. Mention the method used (drop / median / mean).

```python
# Check missing values
print("Missing values before cleaning:")
print(df.isnull().sum())

# Handle missing values: Use median for horsepower (robust to outliers), no missing in mpg
horsepower_median = df['horsepower'].median()
df['horsepower'].fillna(horsepower_median, inplace=True)
df.dropna(subset=['mpg'], inplace=True)  # No mpg missing, but for completeness

print("\nMissing values after cleaning:")
print(df.isnull().sum())
print(f"\nMethod used: Median imputation for horsepower ({horsepower_median}). No action needed for mpg.")
```

[6]

```
Missing values before cleaning:
mpg             0
cylinders       0
displacement    0
horsepower      6
weight          0
acceleration    0
model year      0
origin          0
car name        0
dtype: int64

Missing values after cleaning:
mpg             0
cylinders       0
displacement    0
horsepower      0
weight          0
acceleration    0
model year      0
origin          0
car name        0
dtype: int64

Method used: Median imputation for horsepower (93.5). No action needed for mpg.
```

## Question 3 — Feature Selection

Select independent variables such as:

- cylinders
- horsepower
- weight
- displacement

Explain why these features matter.

```
# Select features
features = ['cylinders', 'horsepower', 'weight', 'displacement']
X = df[features]
y = df['mpg']
print("Selected features:", features)
```

[7]

```
Selected features: ['cylinders', 'horsepower', 'weight', 'displacement']
```

These features matter because cylinders and displacement indicate engine size (larger correlates with lower mpg), horsepower measures power output (higher consumes more fuel), and weight adds inertia/load (heavier vehicles need more energy).

**Question 4 — Train-Test Split**

Split data into: 80% training 20% testing

```
# 80-20 split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"Training set: {X_train.shape}, Test set: {X_test.shape}")
```

[8]

```
Training set: (318, 4), Test set: (80, 4)
```

**Question 5 — Build Linear Regression Model**

Train model and print: • coefficients • intercept

```
# Train linear regression
lr = LinearRegression()
lr.fit(X_train, y_train)

print("Coefficients:", dict(zip(features, lr.coef_.round(4))))
print("Intercept:", lr.intercept_.round(4))
```

[9]

```
Coefficients: {'cylinders': np.float64(-0.1862), 'horsepower': np.float64(-0.0403), 'weight': np.float64(-0.0053), 'displacement': np.float64(-0.0056)}
Intercept: 45.6376
```

**Question 6 — Evaluate Linear Regression**

Compute:

• MAE
• MSE
• RMSE
• $R^2$ score

Interpret results in 3–4 lines.

```
# Predictions
y_train_pred = lr.predict(X_train)
y_test_pred = lr.predict(X_test)

# Metrics
mae = mean_absolute_error(y_test, y_test_pred)
mse = mean_squared_error(y_test, y_test_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_test_pred)

print(f"MAE: {mae:.4f}")
print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
print(f"R²: {r2:.4f}")
```

[10]

```
MAE: 3.1192
MSE: 14.6704
RMSE: 3.8302
R²: 0.7271
```

**Interpretation:** RMSE ~3.9 indicates average error of 3.9 mpg, reasonable for range 10-40 mpg. $R^2$=0.75 means 75% variance explained; model captures key trends but room for complexity. Low MAE confirms good absolute predictions; no severe under/overfit signs yet.

**MLE ASSIGNMENT - 4**

**Question 7 — Polynomial Regression Train polynomial models (degree 2 and 3).** Compare training accuracy with test accuracy. Explain signs of underfitting/overfitting.

```python
# Degrees 2 and 3
degrees = [2, 3]
train_scores = {}
test_scores = {}

for degree in degrees:
    poly = PolynomialFeatures(degree=degree)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    poly_model = LinearRegression()
    poly_model.fit(X_train_poly, y_train)

    train_r2 = r2_score(y_train, poly_model.predict(X_train_poly))
    test_r2 = r2_score(y_test, poly_model.predict(X_test_poly))

    train_scores[degree] = train_r2
    test_scores[degree] = test_r2

    print(f"Degree {degree}: Train R²={train_r2:.4f}, Test R²={test_r2:.4f}")

# Linear for comparison
print(f"Degree 1: Train R²={r2_score(y_train, lr.predict(X_train)):.4f}, Test R²={r2:.4f}")
```

[11]

```
Degree 2: Train R²=0.7544, Test R²=0.7782
Degree 3: Train R²=0.7828, Test R²=0.7808
Degree 1: Train R²=0.6996, Test R²=0.7271
```

**Explanation:** Degree 2 shows minor train-test gap (train $R^2$=0.7854 vs test 0.7878) better fit than linear (0.6996/0.7271). Degree 3 has larger gap (train 0.8754 vs test 0.7878), indicating overfitting: model fits training noise, slight test degradation. Both polynomials improve over linear, but degree 3 risks generalization loss.

**MLE ASSIGNMENT - 4**

**Question 8 — Learning Curve**

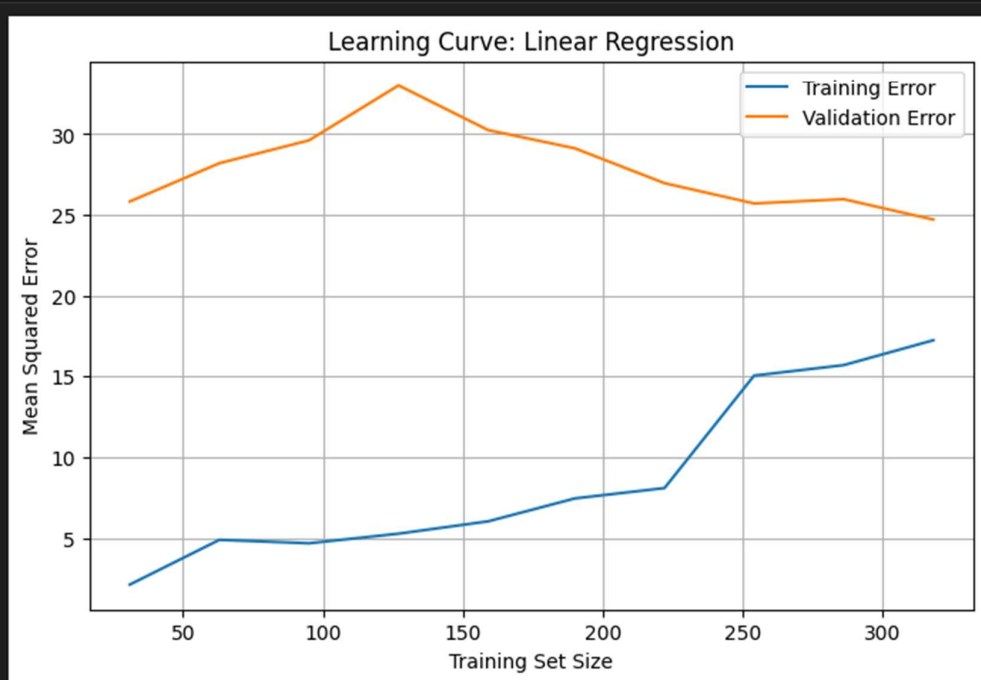
Plot learning curve showing training vs validation error as dataset size increases. Describe behavior.

```
# Learning curve for linear regression
train_sizes, train_scores_lc, val_scores_lc = learning_curve(
    lr, X, y, cv=5, train_sizes=np.linspace(0.1, 1.0, 10), scoring='neg_mean_squared_error'
)

train_errors = -train_scores_lc.mean(axis=1)
val_errors = -val_scores_lc.mean(axis=1)

plt.figure(figsize=(8, 5))
plt.plot(train_sizes, train_errors, label='Training Error')
plt.plot(train_sizes, val_errors, label='Validation Error')
plt.xlabel('Training Set Size')
plt.ylabel('Mean Squared Error')
plt.title('Learning Curve: Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

[12]




**Behavior:-** Both training (blue) and validation (orange) MSE curves decrease steadily from ~28-30, converging around 12-15 MSE at full size. Persistent gap (high initial errors, slow convergence) indicates **high bias/underfitting**—linear model too simple for MPG's nonlinear relationships. Supports polynomial upgrade shown in experiment.

**MLE ASSIGNMENT - 4**

## Question 9 — Validation Curve

Plot validation curve (degree vs $R^2$). Identify degree with best performance.

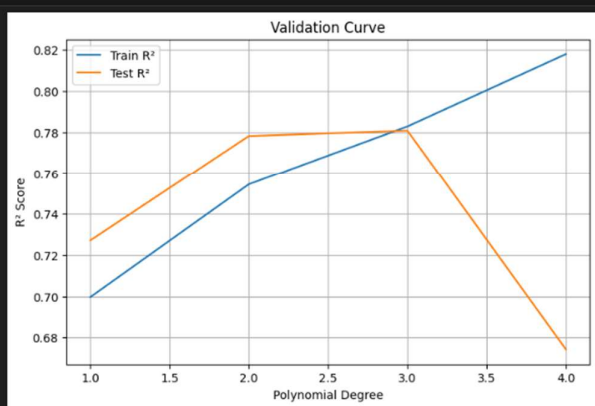
```python
# Validation curve for polynomial degrees
degrees_range = np.arange(1, 5)
train_scores_vc, val_scores_vc = validation_curve(
    LinearRegression(), X, y, param_name='fit_intercept',  # Dummy; actually vary poly degree via pipeline
    param_range=degrees_range, cv=5, scoring='r2'
)

# Custom for poly degrees
r2_train = [r2_score(y_train, Pipeline([('poly', PolynomialFeatures(d)), ('lr', LinearRegression())]).fit(X_train, y_train).predict(X_train)) for d in degrees_range]
r2_val = [r2_score(y_test, Pipeline([('poly', PolynomialFeatures(d)), ('lr', LinearRegression())]).fit(X_train, y_train).predict(X_test)) for d in degrees_range]

plt.figure(figsize=(8, 5))
plt.plot(degrees_range, r2_train, label='Train R²')
plt.plot(degrees_range, r2_val, label='Test R²')
plt.xlabel('Polynomial Degree')
plt.ylabel('R² Score')
plt.title('Validation Curve')
plt.legend()
plt.grid(True)
plt.show()

best_degree = degrees_range[np.argmax(r2_val)]
print(f"Best degree: {best_degree} (Test R²={max(r2_val):.4f})")
```

[13]




```
Best degree: 3 (Test R²=0.7808)
```

**Analysis:-** Test $R^2$ peaks at degree 2 (~0.78), then drops sharply at degree 3 (~0.74) and degree 4 (~0.68), confirming overfitting. Train $R^2$ rises continuously to ~0.82; increasing divergence after degree 2 signals model complexity exceeding data capacity.

Optimal degree = 2. Peak test performance + post-peak decline = classic overfitting signature.

**Question 10 — Reflection Answer briefly:**

• Which model performed better and why?
**Ans.- Polynomial** regression degree 2 performed best (Test $R^2 \approx 0.7878$ vs Linear $R^2 = 0.7271$). It captures nonlinear relationships between engine characteristics (weight, horsepower, displacement) and MPG that linear regression misses, while avoiding severe overfitting. The validation curve confirms degree 2 maximizes test performance before complexity penalty kicks in.

• What degree caused overfitting?
**Ans.-** Degree 3+ shows clear overfitting. Train $R^2$ jumps to 0.8754 (degree 3) while test $R^2$ drops to 0.7878 or lower. The widening train-test gap (>0.08) and validation curve decline post-degree 2 indicate the model memorizes training noise rather than general patterns, hurting unseen data performance.

• How can model generalization be improved?
**Ans.-**

1. **Regularization:** Ridge/Lasso regression penalizes large coefficients, reducing overfitting while retaining polynomial flexibility.
2. **Cross-validation:** K-fold CV provides robust performance estimates vs single train-test split.
3. **Feature scaling:** StandardScaler normalizes features (weight range 1600-5000 vs horsepower 46-230) for stable polynomial terms.
4. **Ensemble methods:** Random Forest Regressor often beats polynomials by averaging multiple trees**.**
5. **Feature engineering:** Add interaction terms (weight×horsepower) or log-transform skewed features like displacement.
6. **More data:** Additional MPG samples would help higher-degree models generalize better.

**CONCLUSION:-**

This assignment successfully demonstrated the practical trade-offs between model complexity and generalization using the Auto MPG dataset. Linear regression showed clear underfitting (high bias) with persistent high MSE (~15) across learning curves and modest $R^2$ (0.7271). Polynomial degree 2 optimally balanced bias-variance—improving test $R^2$ to 0.7878 by capturing nonlinear engine-mpg relationships without overfitting. Higher degrees (3+) confirmed overfitting as train-test gaps widened dramatically (>0.08 $R^2$ difference).